

Code:

```
// Program to find the inverse of a matrix using Gauss-Jordan Elimination
```

```
#include<iostream>
#include<iomanip>
#include<cmath>
using namespace std;

int main()
{
    cout<<fixed<<setprecision(2);

    int i,j,k,n;
    double mat[100][200], t;

    cout<<"Enter the dimension of the matrix: ";
    cin>>n;

    cout<<"Enter the elements of the matrix:\n";
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            cin>>mat[i][j];

    // Form augmented matrix [A | I]
    for(i=0;i<n;i++)
    {
        for(j=n;j<2*n;j++)
        {
            if(j==i+n)
                mat[i][j]=1;
            else
                mat[i][j]=0;
        }
    }

    cout<<"\nAugmented Matrix:\n";
    for(i=0;i<n;i++)
    {
```

```

for(j=0;j<2*n;j++)
    cout<<setw(8)<<mat[i][j];
cout<<endl;
}

```

// Partial Pivoting

```

for(i=0;i<n;i++)
{
    for(k=i+1;k<n;k++)
    {
        if(fabs(mat[k][i])>fabs(mat[i][i]))
        {
            for(j=0;j<2*n;j++)
            {
                double temp=mat[i][j];
                mat[i][j]=mat[k][j];
                mat[k][j]=temp;
            }
        }
    }
}

```

// Gauss-Jordan Elimination

```

for(i=0;i<n;i++)
{
    if(mat[i][i]==0)
    {
        cout<<"\nThe matrix is not invertible.\n";
        return 0;
    }

    for(j=0;j<n;j++)
    {
        if(j!=i)
        {
            t=mat[j][i]/mat[i][i];

            for(k=0;k<2*n;k++)

```

```

        mat[j][k]-=t*mat[i][k];
    }
}

// Make pivot elements equal to 1
for(i=0;i<n;i++)
{
    t=mat[i][i];

    for(j=0;j<2*n;j++)
        mat[i][j]/=t;
}

cout<<"\nInverse Matrix:\n";

for(i=0;i<n;i++)
{
    for(j=n;j<2*n;j++)
        cout<<setw(8)<<mat[i][j];
    cout<<endl;
}

return 0;
}

```

Output:

Enter the dimension of the matrix: 2

Enter the elements of the matrix:

1 2

3 4

Augmented Matrix:

1.00 2.00 1.00 0.00

3.00 4.00 0.00 1.00

Inverse Matrix:

-2.00 1.00

1.50 -0.50

Process exited after 5.315 seconds with return value 0

Press any key to continue . . .